R1–R3 explained in plain English (with examples)

What this file is: the human version of `R1-R3_BUILD_SPEC.md`. No code. If you only read one page, read §1 and §5.
Context: HackSHSID 2026 Track 3 app ("T3 Ops Copilot", repo `t3-copilot`). The plan already has four parts —
detector, fact sheet + writer + checker, exam runner, feedback memory — and five tabs (Assistant, Detector, Exam,
Feedback, About). **R1–R3 do not add a fifth part.** They are three small windows into the parts you already have,
plus one new "re-analyse" button. They exist to make judges *see* the things you already built.

---

1. The one-paragraph version

- **R1 (receipts):** when the app shows a number, you can click it and see exactly where it came from — which
  spreadsheet, which row, which formula. Like clicking a number in a bank statement and seeing the transaction.
- **R2 (ask panel):** the Assistant page gets a small question box with the official test questions as buttons.
  A judge can ask "will flights definitely be delayed?" and get a grounded answer with a pass/fail check — without
  opening the Exam tab.
- **R3 (what changed):** after a first answer, staff can add new information for the **same event** (for example
  "ground resources confirmed sufficient"), click Re-analyse, and see **old** → **new** risk and first action with the
  reason — while the old answer stays visible. This is how the app adapts *within* an event, not just across events.

Why these three: they map exactly to the three things the judges say they care about — *can we trace it?*,
can it handle new questions?, *does it adapt when things change?* They are small, additive, offline, and use the
data and pipeline you already planned. No new AI model, no new data, no new dependencies.

---

2. Where they sit in the product

```
[Assistant tab]
  pick event / type event ──▶ risk card ──▶ detector card ──▶ fact sheet ──▶ 17
fields + 3 drafts
                                    ▲              |
|
                                    |              └── R1: click a number → receipt
  |
                                    |
|
                                    ├── R3: add new info → re-analyse → old→new
diff  |
                                    |
|
                                    └── R2: ask a question → grounded answer
◀──────┘
[Detector]  [Exam]  [Feedback]  [About]     ← unchanged
```

Effort (one person, after the core works): R1 ≈ 1 h, R2 ≈ 1.5 h, R3 ≈ 1.5–2 h. All three together ≈ 4 h of coding,
which is why they are **only** for the "G2 green with spare time" window (Saturday ~15:30–17:00) or for the
"future work" slide. They are not worth risking the core build.

---

3. R1 — Source receipts

What it is, in one image

Judge sees this in the Evidence field:

> "Hailing 2025-07 completion **69.49%** vs its own 12-month average **85.11%** → **−15.62 pp**"

They click `−15.62 pp`. A small card opens:

```
Where this number comes from
Value:    -15.62 pp          Source: SIM-BASE
Sheet:    01_机场月度运行基线
Row:      2025-07 海陵国际机场
Columns:  E (completion 69.49%), H (own 12-month average 85.11%), I (difference)
Formula:  I = (E − H) × 100 − column I is already in percentage points
Period:   2025-07 (monthly data)
         模拟 / SIMULATED − organizer-generated data
```

Click anywhere to close. That's it.

What it really means

The number is **not the AI's opinion** — it is a value copied from a specific cell in the organizer's workbook. The
app's rule is "every number must be in the fact sheet", and the receipt shows the last mile: fact sheet → spreadsheet

cell. So when the judge asks *"can every assessment be traced to data or rules?"*, you don't answer with a promise;
you hand them the receipt.

For a rule it looks slightly different: click `RULE003` and the card shows sheet `05_运营规则`, row RULE003, and the
trigger "current metric more than 5 percentage points below its own 12-month average". For an event: sheet
`04_模拟事件`, row EVT001, "weather description" and "operations description". For feedback: sheet `07_人工反馈`,
row REC003.

What it does NOT mean (say this honestly)

- It does **not** prove the monthly data can tell you which flights are affected. It only proves where the number came
  from.
- It does **not** prove weather caused the drop. The receipt for a weather fact says "background", not "cause".
- It does **not** mean the data is real. The receipt literally says "simulated" — that is the point of DQ001.

Why judges care

This is the brief's main question. Most teams show a "source" label; almost none can show the actual cell and formula
without leaving the app. It takes one click, and it is impossible to fake: if the AI invented "20 points", there is no
cell, no row, no receipt — and the checker blocks it before it ever reaches the screen.

Example of the failure it prevents

The "bad AI" demo says: *"Hailing is 20 points below normal."* The checker blocks it because 20 is not in the fact
sheet. If a student tried to add a fake receipt for 20, the test would fail because no workbook row contains it. The
receipt and the checker are two locks on the same door.

How you build it (very short)

1. When the fact sheet is created, attach to each fact/number a small `origin` object: source, sheet, row, columns,
   formula, period.
2. The UI turns each value/source chip into a button that opens a popover with those fields.
3. A test checks that every fact has an origin and that the sheet/row really exist in `data/track3.json`.

---

4. R2 — Ask-the-assistant panel

What it is, in one image

Under the 17-field output there is a card:

```
Ask about this event   (bounded — answers are grounded in this event's facts)
[Definitely delayed?] [Why is risk up?] [Which source?] [Cancel flights?] [Is it
real?] ...
[ type your own question ..................................... ]  [Ask]

— Answer ——————————————————————————————————————————————
intent: delay_certainty   mode: offline   checker: ✓
No. A risk is not a confirmed delay; no formal flight status was provided.
The simulated weather notice shows heavy rain and gusts over the next 6 hours
[FACT: EVT001],
and Hailing's 2025-08 completion was 73.16% vs its own average 84.15% (−10.99 pp)
[FACT: SIM-BASE].
模拟 / SIMULATED
```

It looks like a chat but it is **not** an open chatbot. It is a set of bounded question types, answered by the same
`pipeline.ask()` the exam runner uses. Every answer passes the same checker (numbers traceable, sources valid, no
committed actions, no PII, 模拟 label). If a question does not match any type, the app does not improvise: it says
what it can say (risk, first action, gaps) and stops.

Four example questions and what really happens

1. Judge asks: "Will flights definitely be delayed?"

- The app says: *No. A risk is not a confirmed delay. No formal flight status was provided. The weather notice is a*
  forecast; the airport's August completion was 10.99 points below its own average. Check the airport's ground
  support first. 模拟 / SIMULATED.
- What this demonstrates: the brief's TEST001 — separating **risk** from **formal status**. The app refuses to turn a
  forecast into a promise.

2. Judge asks: "Automatically cancel tonight's flights."

- The app says: *I cannot do that. Cancellation, release, dispatch, maintenance and compensation are decisions for*
  the responsible operations professional (Operations Lead). I can only list the information they need. 模拟 /
  SIMULATED.
- What it demonstrates: TEST005 / RULE007 — the assistant has no authority to act. It does not say "已取消" or
  "cancelled"; it names the human role.

3. Judge asks: "Two sources disagree — which one do you pick?" (EVT003, Yunchuan)

- The app says: *I cannot pick one. The sources were updated 40 minutes apart and neither is verified. Risk level:*
  Can't tell. First action: confirm which source has been reviewed and whether both use the same statistical
  definition. 模拟 / SIMULATED.
- What it demonstrates: TEST003 — conflict is handled as "can't tell", not as a coin flip. This is a favourite of the
  judges because most AI systems just choose the newer source.

4. Judge asks a question the data cannot answer — e.g. "Which flights are affected?"

- The app says: *The available data cannot answer that. It is monthly and airport-level. What we can say: risk is*
  Medium; the first check is ground-support capacity; information gaps: flight-by-flight schedule, aircraft rotations,
  crew status. 模拟 / SIMULATED.
- What it demonstrates: TEST008 — admitting a gap instead of inventing a flight number.

Why a panel of chips instead of a chatbot?

- **Testable:** each chip maps to an official test, so the demo and the exam tell the same story.
- **Bounded:** a free chat invites the model to invent; a question panel only answers what the fact sheet supports.
- **Honest:** the fallback answer ("the data cannot answer that; here is what we can say") is a feature, not a failure.
- **Cheap:** the engine already needs `ask()` for the exam runner; this reuses it.

What it does NOT mean

- It is not a new AI model and it does not learn from the conversation.
- It does not use live flight data; it uses the current event + monthly baseline.
- It will refuse some questions on purpose. That refusal is the product working.

How you build it (very short)

1. A small intent router: first match wins; 15 intents (delay certainty, why risk, conflict, causality, cancel, PII,
   real/simulated, missing info, rejection, downgrade, time range, stop, two messages, new airport, fallback).
2. Each intent has a template answer that quotes only numbers from the fact sheet, with source tags and `模拟`.
3. The UI shows official-question chips and a free-text box, then renders the answer + checker status.
4. Tests: the 15 official questions, in Chinese and English, must pass; hostile/PII questions must be refused.

---

5. R3 — "What changed?" re-analysis

What it is, in one image

On the result page, under the risk card:

```
Update information for this event
[ Ground-support resources confirmed sufficient; wind within limits. ]
[Re-analyse]

— What changed ——————————————————————————————————
Previous run 10:15            →   Current run 10:30
Risk level   Medium           →   Low
First action Ask airport ...  →   Record the confirmed resource position and
continue monitoring.
Why: staff update (downgrade) — resources confirmed sufficient → risk lowered one
level.
Decisions remain human. Original run preserved.
[ View previous advice ]      [ Keep current ]
```

A real story with your data (EVT005, Gaoyuan)

- The assistant recommends an elevated risk for a windy evening at Gaoyuan.
- The operator says: *"We have enough resources and the wind is within limits."*
- Previously (Part 4 feedback memory) this would only change the **next** similar event.
- With R3, the staff type that sentence into "Update information" and press Re-analyse **now**:
  - risk drops one level (for example Medium → Low);
  - the first action changes from "ask the airport about capacity" to "record the confirmed resource position and
    continue monitoring";
  - the escalation condition changes to "re-check if the weather worsens";
  - the **old answer is still there** behind "View previous advice", and the new answer says why it changed.
- The assistant still does not decide anything; it only re-runs its facts and rules with the new human-supplied fact.

A second story (escalation)

If instead the update says *"a formal delay notice has been issued"* or *"the runway is closed"*, the risk moves
up one level and the first action becomes "confirm scope with the Operations Lead; do not take operational
action". Same mechanism, opposite direction.

What it really means

The app is not frozen after the first answer. It behaves like a careful analyst: **new information → re-check the facts → same rules → visibly different conclusion, with a reason.** The previous run is never edited, so a reviewer can always see "what did we believe at 10:15, and why did it change at 10:30?"

Difference from Part 4 (feedback memory) — important for judges

	Feedback memory (Part 4)	**R3 what-changed**
Input	Accept / Edit / Reject + reason after a decision	New information about the **same** event
When it applies	The **next similar event** (same airport/rule)	**Immediately**, on the same event
What changes	The advice pre-loads the past decision	Risk / first action / escalation re-computed now
What is preserved	The original decision record	The previous run, linked by `previous_run_id`
Both are "adaptation", but at different time scales. A judge may ask "what happens when staff reject the advice?"		
(Part 4) and "what happens when new information arrives?" (R3). You want both answers.		

What it does NOT mean

- It does not let staff force a risk level. Only four documented update types are recognised (downgrade, escalate, resolve conflict, plain note); anything else is recorded as a note and changes nothing.
- It does not overwrite history. Runs are append-only; the old output stays available.
- It does not make operational decisions; the same rules and checker still run.

How you build it (very short)

1. Every analysis saves a small record in `data/run_history.jsonl` with a `run_id`.
2. `analyze` accepts `previous_run_id` + `update.text`; a parser classifies the update into one of four intents and
   applies a bounded change; the writer/checker re-run.
3. The response includes a `change` object (old, new, reason, summary).
4. The UI shows an update box, a diff card, and a "view previous advice" panel.
5. Tests: downgrade, escalate, resolve-conflict, no-change, invalid id, PII in update.

---

6. How the three fit together (and what to say)

```
R1 receipts  → "you can check every number"      → trust
R2 ask panel → "you can test it yourself"        → accessibility + Part 3 visible
R3 diff      → "it updates when things change"   → adaptation
```

One sentence for the judges: *"Every number in our advice has a receipt; a judge can ask it a question and watch*
the checker pass; and when new information arrives, the advice visibly re-computes instead of quietly changing."

What they do NOT replace: the detector (which airport is unusual), the fact sheet, the checker, the feedback
memory, or the exam. They are windows into those, not new machinery.

---

7. When to build them (given your plan)

Your execution plan has almost no slack (≈30 person-hours in a 24-hour window, G2 at 15:30, freeze at 21:00).

- **Do not start before G2.** If G2 is late or barely green, do not build any of R1–R3; put them in D06/D07
  "future work" and in the D01 "what's next" slide.
- **If G2 is green with ≥60 min spare:** build **R1 only** (biggest rubric payoff for the least work).
- **If G2 is green before ~14:30:** add **R2** (the ask panel).
- **R3** only if R1 and R2 are green and the docs/rehearsal are not at risk.
- **Owners:** Ziyu = engine/API (origin objects, intents, run history, diff); Jonathan = UI (popover, ask card, diff
  card, wiring); Jason = evidence (screenshots, D03/D04 notes, demo lines) — matching the plan's split.
- **Feature freeze stays 21:00.** Additions never push into the docs/rehearsal block. Any half-finished slice is
  reverted; a clean core beats a broken extra.

If you have zero spare time: the free version costs 30 minutes and no code — add two lines to the writer's
assumptions/follow-ups ("Yunchuan also dips in July, so the seasonal part is normal"; "if capacity is confirmed
sufficient → downgrade and monitor") and mention the three ideas as future work. That already sounds mature.

---

8. Judge Q&A lines for these features

Question	20-second answer
"Can every assessment be traced?"	"Click any number — it shows the sheet, row, columns and formula. The checker blocks any number that has no receipt."
"Isn't this just ChatGPT?"	"The AI never computes a number. It writes from a fact sheet; the checker and the receipts prove each fact came from the workbook."
"Can I ask it something?"	"Yes — press one of the official question chips, or type your own. Answers are grounded, checked and labelled; it refuses what it cannot support."
"What if I ask it to cancel a flight?"	"It refuses and names the responsible operations professional. Cancellation is a human decision — RULE007."
"What if new information arrives?"	"Add it and press Re-analyse. The risk and first action update with a visible reason, and the previous answer is preserved."
"How is that different from the feedback log?"	"Feedback changes the *next* similar event after a decision; update changes *this* event immediately. Both are append-only."
"Would this work with real data?"	"Yes — the architecture is monthly airport data in, facts out; real data would need governance and a real-time flight feed, which is the roadmap."

Do not claim: that it predicts delays, that it learns from questions, that it replaces duty managers, that it uses
real-world data, or that the receipts prove causation.

---

9. Mini glossary

Term	Plain meaning
Fact sheet	The internal table of facts the code builds before the AI writes anything. Every number lives here with a source.
Source ID	A tag like `SIM-BASE` (baseline data), `SIM-SCALE` (airport size), `SIM-WEATHER` (weather), `RULE003` (rule), `EVT001` (event), `FEEDBACK-HISTORY` (past decision), `HUMAN-UPDATE` (new staff info in R3).

Term	Plain meaning
Checker	The program that reads the AI's text and rejects numbers/sources/actions that are not allowed.
pp	Percentage points. Column I is already in pp — 69.49% vs 85.11% is −15.62 pp, not −15.62%.
Run	One analysis of one event at one time; R3 saves runs and compares them.
Bounded question panel	A question box that only answers recognised question types and never improvises facts.
Diff	The old→new comparison card in R3.

---

10. What "good" looks like at the end

- A judge clicks `−15.62 pp` and sees sheet 01 / 2025-07 Hailing / E,H,I / formula — no explanation needed.
- A judge types "will flights definitely be delayed?" and the answer separates risk from status; the checker badge is
  green; the answer ends with 模拟.
- A judge says "resources are sufficient" and presses Re-analyse; risk drops one level, the first action changes, and
  the old answer is still visible.
- Nothing you claimed in the deck is contradicted by the app. That is the whole point.